Лабораторная работа 1. Задание 1: множество Мандельброта

1. Постановка задачи

Написать параллельную программу (pthreads), которая находит заданное количество точек множества Мандельброта и записывает их координаты в CSV-файл. Запуск: ./program nthreads npoints, где nthreads — число потоков, npoints — число точек.

2. Сведения о реализованном алгоритме

Число $c = x + iy$ принадлежит множеству Мандельброта, если последовательность $z_1 = 0$, $z_{n+1} = z_n^2 + c$ ограничена; достаточно проверить, что $|z_n| < 2$ для всех n. На практике делается MAX_ITER = 1000 итераций: если за это время |z| не достиг 2, точка считается принадлежащей множеству. Комплексные числа раскладываются на действительную и мнимую части: $Re = x^2 - y^2 + cx$, $Im = 2xy + cy$; вместо $|z| \geq 2$ проверяется $x^2 + y^2 \geq 4$ (без извлечения корня).

Всё множество лежит в прямоугольнике $Re \in [-2; 0.5]$, $Im \in [-1.25; 1.25]$. Программа выбирает случайные точки из этого прямоугольника и оставляет те, что прошли проверку, пока не наберёт нужное количество.

Распараллеливание:

- Точки делятся между потоками поровну; каждый поток ищет свою часть независимо от других.
- У каждого потока свой массив для точек и свой генератор случайных чисел (my_drand со своим seed), поэтому общих изменяемых данных нет и мьютекс не нужен.
- Запись в CSV выполняет главный поток после pthread_join; в замер времени она не входит.

3. Пояснения к коду

Программа состоит из файла mandelbrot.c и вспомогательных файлов из материалов курса: timer.h (макрос GET_TIME для замера времени) и my_rand.c / my_rand.h (потокобезопасный генератор случайных чисел). Сборка: gcc -O2 -Wall -pthread mandelbrot.c my_rand.c -o program.

Общие переменные.

- thread_count, npoints — число потоков и число точек из аргументов командной строки.
- counts[i] — сколько точек должен найти поток i. Вычисляется в main до запуска потоков: npoints / thread_count, и первые npoints % thread_count потоков получают на одну точку больше, чтобы в сумме вышло ровно npoints.
- points[i] — отдельный массив потока i, куда он записывает найденные точки подряд: x_0, y_0, x_1, y_1, … Память под все массивы выделяется в main заранее.

Потоки только читают thread_count, counts и указатели points, а пишут каждый в свой массив, поэтому одновременный доступ к одним и тем же данным на запись невозможен и синхронизация не требуется.

Функция In_mandelbrot(cx, cy) проверяет одну точку c = cx + i·cy. Переменные x, y хранят z (вначале z = 0). На каждой итерации вычисляется $z^2 + c$ по формулам $Re = x^2 - y^2 + cx$, $Im = 2xy + cy$. Если $x^2 + y^2 \geq 4$ (то есть $|z| \geq 2$), функция сразу возвращает 0 — точка не принадлежит множеству. Если за MAX_ITER = 1000 итераций этого не произошло, возвращается 1.

Функция потока routine(rank). Номер потока передаётся через аргумент: long my_rank = (long) rank. Поток берёт свою норму counts[my_rank] и свой массив points[my_rank], заводит собственное состояние генератора seed = my_rank + 1 (у всех потоков разное, поэтому они генерируют разные последовательности точек) и в цикле:

1. получает случайную точку прямоугольника: cx = X_MIN + (X_MAX − X_MIN) · my_drand(&seed), аналогично cy (my_drand возвращает число от 0 до 1);
2. проверяет её функцией In_mandelbrot;
3. если точка принадлежит множеству — записывает её в свой массив и увеличивает счётчик найденных.

Цикл заканчивается, когда найдено counts[my_rank] точек.

Для генерации используется my_drand, а не стандартная rand(): у rand() одно скрытое состояние на всю программу, и при вызове из нескольких потоков возникает гонка. У my_drand состояние хранится в переменной seed, которую передаёт вызывающий, — у каждого потока своя.

Функция main.

1. Читает аргументы через strtol и проверяет, что они положительные.
2. Выделяет память под дескрипторы потоков, массив counts и массивы points.
3. Засекает время GET_TIME(start), создаёт потоки pthread_create(&thread_handles[i], NULL, routine, (void*) i) и ждёт их завершения pthread_join; затем GET_TIME(end) и печатает время end − start на экран.
4. После завершения всех потоков записывает файл mandelbrot.csv: строка заголовка x,y, далее точки всех потоков по очереди. Запись в файл последовательная и выполняется вне замера времени, так как измеряется именно параллельная часть.
5. Освобождает выделенную память.

Вывод программы: на экран — время вычислений в секундах; в файл mandelbrot.csv — npoints строк с координатами точек множества.

4. Условия измерений

- Процессор: Intel Core i5-1335U, 10 ядер (2 производительных + 8 энергоэффективных), 12 потоков.
- Время измерялось макросом GET_TIME; каждая конфигурация (npoints = 200000) запускалась 5 раз, бралось минимальное время.

5. Результаты замеров

Ускорение $S = T_1 / T_p$, эффективность $E = S / p$, где T_1 — время на 1 потоке, T_p — на p потоках.

Потоки p	Запуски, с	Время T (минимум), с	Ускорение S	Эффективность E
1	1.030; 1.125; 1.075; 1.063; 1.344	1.030	1.00	1.00
2	0.566; 0.591; 0.779; 0.647; 0.613	0.566	1.82	0.91
4	0.396; 0.388; 0.370; 0.384; 0.371	0.370	2.79	0.70
8	0.219; 0.235; 0.232; 0.241; 0.228	0.219	4.71	0.59

Каждая конфигурация запускалась вручную 5 раз командой ./program p 200000; в столбце «Время» — минимальное из пяти значений. Ускорение и эффективность рассчитаны по минимальным временам.

6. Выводы

Программа корректно находит заданное количество точек множества Мандельброта и записывает их в CSV-файл.

С ростом числа потоков время работы уменьшается: на 2 потоках программа работает в 1.82 раза быстрее, чем на одном (эффективность 0.91), на 4 — в 2.79 раза (0.70), на 8 — в 4.71 раза (0.59). Хорошее ускорение объясняется тем, что потоки независимы: у каждого свой массив точек и свой генератор случайных чисел, поэтому синхронизация во время вычислений не нужна, а случайный выбор точек равномерно распределяет нагрузку.

Эффективность при этом снижается с ростом числа потоков. Основная причина — устройство процессора i5-1335U: из 10 его ядер только 2 производительных (P-ядра), остальные 8 — более медленные энергоэффективные E-ядра. Один-два потока операционная система запускает на быстрых P-ядрах, а при 4 и 8 потоках часть работы попадает на E-ядра, и потоки, получившие медленные ядра, заканчивают позже остальных. Кроме того, ноутбучный процессор снижает тактовую частоту, когда загружено много ядер одновременно (ограничение по мощности и нагреву), а также сказываются накладные расходы на создание и завершение потоков. Разброс времени между запусками (например, 1.03–1.34 с на одном потоке) вызван фоновыми процессами операционной системы и изменением частоты процессора, поэтому для анализа использовалось минимальное время из пяти запусков.